BVS1 · Winter Semester 2026/2027 · TH Köln · ICCT

Exercise 1 — From Source to First Instructions

Planned time: 90 minutes

Guiding Question

> How does our RV64 program get from source code to its first executed instructions — and what do those instructions actually do?

What You Should Be Able to Do Afterwards

You should be able to:

- reconstruct the physical-host → development-VM → QEMU → RV64-target layers;
- distinguish virtualization from emulation using the ISA;
- explain source → object files → linked ELF → target memory;
- distinguish symbols an object file provides from symbols it cannot yet resolve;
- read and explain the small startup program in `start.S`;
- make one small, observable change that crosses the assembly/C boundary.

Starting Point — Lab 0 Catch-Up

Use the first **20 minutes** to establish a common starting point.

If Lab 0 is not complete, continue with the Lab 0 sheet and ask for help where needed.

If your environment is ready, verify:

```
multipass shell bvs1
cd ~/teaching-bvs1-code
make check
make run
```

You should see:

```
Hello BVS1
```

Terminate QEMU with **Ctrl-a**, release the keys, then press **x**.

After 20 minutes, continue with the exercise even if setup is not yet complete. Weekly lab sessions remain available for setup and checkpoint support.

1. Locate the Target

Arrange these layers from outside to inside:

```
Hello BVS1
QEMU process
physical host
RV64 target machine
Ubuntu development VM: bvs1
```

Add these relations:

```
hosts via virtualization
runs
emulates
executes
```

Use the verbs deliberately:

- the **physical host hosts the Ubuntu development VM via virtualization**;
- the **Ubuntu development VM runs the QEMU process**;
- **QEMU emulates the RV64 target machine**;
- the **RV64 CPU executes the RV64 instructions** of `Hello BVS1`.

Then answer:

1. Where does QEMU run?
2. Which machine executes the RV64 instructions?
3. Does that target contain a guest operating system at this point?
4. Why is the Ubuntu development VM an example of virtualization, while our RV64 machine in QEMU is an example of emulation?

Use one test for the last question:

> **Can the physical CPU execute the guest / target instruction directly?**

2. Follow the Build — and the Symbols

Reconstruct the path from source to execution:

```
main.c
start.S
main.o
start.o
linker.ld
hello-bvs1.elf
target memory
RV64 CPU
```

Add:

```
compile
assemble
link
load
```

```
execute
```

The distinction matters:

- the linker produces `hello-bvs1.elf`;
- QEMU **loads** that ELF into the emulated target's memory;
- the RV64 CPU then **executes** its instructions, beginning at the entry point `_start`.

Now inspect the two object files. If `build/manual/start.o` and `build/manual/main.o` are missing, repeat the corresponding compile/assemble steps from Lecture 2 first.

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

A typical result contains:

```
start.o:
                 U _stack_top
0000000000000000 T _start
                 U main

main.o:
000000000000.... T main
...
```

Interpret only the symbols `_start`, `main`, and `_stack_top`.

For each one, decide:

- which object file **provides** the symbol;
- which object file **references it but cannot yet resolve it**;
- or whether it is provided somewhere else.

Then locate the missing source of `_stack_top` in `linker.ld`.

Explain in one sentence why the linker is needed even though both object files already contain RV64 machine code.

3. Read Your First RISC-V Assembly

Open `start.S`:

```
_start:
    la sp, _stack_top
    call main
1:
    j 1b
```

You only need the following vocabulary:

```
sp            stack pointer
la            load an address
```

```
call          transfer control and remember how to return
j             jump
1:            local label
1b            the nearest label 1 backward
```

Trace the program from top to bottom.

Answer:

1. Which symbol tells the linker where execution begins?
2. What address is loaded into `sp`?
3. Does `la sp, _stack_top` load the contents of the stack or an address?
4. What happens after `call main`?
5. If `main` returned, what would `j 1b` do?
6. Why must the stack pointer be established **before** calling ordinary C code?

Do not consult the full RISC-V register set or calling convention. Everything needed for this task is above.

4. Make the Assembly/C Boundary Observable

We currently enter C with:

```
call main
```

Add a small C function to `main.c`, after `uart_puts` and before `main`:

```
void before_main(void)
{
    uart_puts("before main\n");
}
```

Then change only the startup sequence:

```
_start:
    la sp, _stack_top
    call before_main
    call main
1:
    j 1b
```

Before rebuilding, predict the output order.

Then rebuild and run:

```
make run
```

Compare prediction with observation.

Afterwards inspect:

If `|` or `grep -E` is unfamiliar, find out what they do before continuing. You should be able to explain how this command reduces the output of `nm`.

```
riscv64-unknown-elf-nm build/hello-bvs1.elf \
  | grep -E '(_start|before_main|main|_stack_top)'
```

Explain:

1. where `before_main` was defined;
2. where it was referenced;
3. why the linker can connect the two;
4. why this experiment does **not** yet create a process.

5. Toward a Process

Our current startup still gives us only one execution:

```
_start
  ↓
before_main
  ↓
main
  ↓
for (;;) {}
```

Suppose an operating system should be able to:

```
run
 ↓
yield
 ↓
OS regains control
 ↓
resume
```

Answer without implementation details:

1. What is the difference between **resume** and **restart**?
2. Why is the startup sequence above not sufficient for resume?
3. Which question must we answer next if we want to build our first process?

Exit Question

Explain this path in your own words:

> **How do `linker.ld`, `start.S`, and `main.c` cooperate so that an RV64 CPU can begin executing C code?**

Use the terms **symbol**, **stack pointer**, **entry point**, **linker**, and **target**.